Runwise Developer Guide

Stack

Prerequisites

- Node.js 22 or later
- npm

Setup

```
git clone https://github.com/alanwaddington/runwise.git
cd runwise
npm install
npm run dev
```

Open http://localhost:5173.

Cleaning up extraneous WASM fallback packages (optional)

npm install/npm ci always installs 5 extra packages — @emnapi/core, @emnapi/runtime, @emnapi/wasi-threads, @napi-rs/wasm-runtime, @tybys/wasm-util — even though npm ls reports them as extraneous and neither npm ci nor npm prune remove them. This is a known upstream npm limitation: they're the WASM32-WASI fallback runtime bundled with @tailwindcss/oxide and @rolldown/binding (Vite's bundler), and npm doesn't correctly skip them via its cpu platform gating when they're nested this way. They're devDependencies-only, unreferenced anywhere in source, and have zero footprint in the deployed Vercel build — safe to ignore.

If you'd like to reclaim the ~9.5MB anyway:

```
npm run clean:wasm
```

This is optional, local-only, and does not change package.json or the lockfile — you'll need to re-run it after every fresh npm install/npm ci if you want to keep node_modules lean.

Project Structure

```
src/
% % %  app.css              # Global styles + Tailwind theme tokens
% % %  app.html             # SvelteKit HTML shell
% % %  lib/
%    % % %  affiliates.ts    # Affiliate product definitions per route (Amazon Associates,
Garmin, and direct/non-affiliate links)
%    % % %  seo.ts           # SEO metadata map (PAGES), sitemap config, OG images
%    % % %  components/      # Shared UI components
%    %    % % %  AdUnit.svelte              # Consent-gated Google AdSense ad unit
%    %    % % %  AdUnit.test.ts
%    %    % % %  AffiliateLinks.svelte      # Per-route affiliate product cards (Amazon,
Garmin, or direct links)
%    %    % % %  AffiliateLinks.test.ts
%    %    % % %  CollapsibleField.svelte    # Generic animated show/hide wrapper (max-height/
opacity, aria-hidden, inert)
%    %    % % %  CollapsibleField.test.ts
%    %    % % %  CookieBanner.svelte        # GDPR cookie consent banner (fixed bottom)
%    %    % % %  CookieBanner.test.ts
%    %    % % %  EducationalSection.svelte  # Homepage "training fundamentals" / "common
mistakes" / "how Runwise helps" sections
```

```
 %    %    % % %  HeroSection.svelte
 %    %    % % %  HeroSection.test.ts
 %    %    % % %  IconWarning.svelte         # Shared inline warning-triangle SVG icon
 %    %    % % %  IconWarning.test.ts
 %    %    % % %  InputField.svelte
 %    %    % % %  InputField.test.ts
 %    %    % % %  PageExplainer.svelte       # Per-route "About this tool" footer content,
driven by lib/content/explainers.ts; sections may include outbound reference links
 %    %    % % %  PageExplainer.test.ts
 %    %    % % %  ResultDisplay.svelte
 %    %    % % %  ResultDisplay.test.ts
 %    %    % % %  SeoHead.svelte            # Per-page meta tags, OG, JSON-LD, AdSense account
verification
 %    %    % % %  SeoHead.test.ts
 %    %    % % %  SiteFooter.svelte          # Footer with Privacy Policy link and Manage
Cookies button
 %    %    % % %  SiteFooter.test.ts
 %    %    % % %  SiteNav.svelte
 %    %    % % %  SiteNav.test.ts
 %    %    % % %  Toast.svelte              # App-wide success/failure notification, driven by
stores/toast.ts (bottom-of-viewport, auto-dismissing)
 %    %    % % %  Toast.test.ts
 %    %    % % %  ToolCard.svelte
 %    %    % % %  ToolCard.test.ts
 %    %    % % %  ToolIcon.svelte
 %    %    % % %  ToolLayout.svelte
 %    %    % % %  ToolLayout.test.ts
 %    %    % % %  WorkoutProfileChart.svelte # Segment-by-segment bar chart (warm-up/work/
recovery/cool-down) for a single workout
 %    %    % % %  WorkoutProfileChart.test.ts
 %    % % %  config/
 %    %    % % %  toolValidation.ts    # Per-field validation rule config shared across tool
pages
 %    % % %  content/
 %    %    % % %  explainers.ts        # Per-route PageExplainer content (heading/intro/
sections, optional outbound links)
 %    % % %  stores/           # Svelte stores for cross-component state
 %    %    % % %  consent.ts               # GDPR consent read/write (localStorage)
 %    %    % % %  consent.test.ts
 %    %    % % %  consentBannerVisible.ts  # Writable store: true = show banner
 %    %    % % %  toast.ts                 # Writable store + showToast()/dismissToast() driving
Toast.svelte
 %    % % %  utils/            # Pure utility modules (no Svelte dependency)
 %    %    % % %  pace.ts                  # Pace/speed conversion functions
 %    %    % % %  pace.test.ts
 %    %    % % %  race-predictor.ts        # Riegel formula, time parsing/formatting, prediction
table
 %    %    % % %  race-predictor.test.ts
 %    %    % % %  race-result-params.ts    # Serializes/parses a race result to/from URL query
params, shared between /training-paces and /workouts
 %    %    % % %  race-result-params.test.ts
 %    %    % % %  training-paces.ts        # VDOT calculation (Daniels' formula), training zone
pace derivation
 %    %    % % %  training-paces.test.ts
 %    %    % % %  hr-zones.ts              # Max HR zones, Friel LTHR zones, LTHR sub-zones,
Tanaka age estimate, calculateDanielsLthrZones (E/M/T/I/R HR zones w/ confidence tiers)
 %    %    % % %  hr-zones.test.ts
 %    %    % % %  vo2max.ts               # ACSM normative data, getFitnessCategory,
getAcsmTable, CATEGORY_COLOURS
 %    %    % % %  vo2max.test.ts
 %    %    % % %  parkrun.ts              # Reference distance list, Riegel prediction, split
generation, PB comparison, WMA age grading
 %    %    % % %  parkrun.test.ts
 %    %    % % %  validation.ts            # Shared input validation helpers (validatePositive,
validateRange)
 %    %    % % %  power-zones.ts           # Per-device (Stryd/Garmin/COROS/Polar) running-power
zone tables and calculatePowerZones()
 %    %    % % %  power-zones.test.ts
 %    %    % % %  workouts.ts              # Pace-mode workout generation (Daniels weekly-
mileage scaling), buildWorkoutsResult, roundWorkoutSegments, sumSegmentMinutes, the
Workout.pattern field
 %    %    % % %  workouts.test.ts
 %    %    % % %  power-workouts.ts        # Power-mode equivalent of workouts.ts (device power !’
estimated pace !’ same session shapes)
 %    %    % % %  power-workouts.test.ts
 %    %    % % %  hr-workouts.ts           # HR-mode equivalent of workouts.ts — duration-based
```

```
(no distance), buildHrWorkoutsResult, falls back to a default pace when no race result
is available
 %    %    % % %  hr-workouts.test.ts
 %    %    % % %  workout-patterns.ts      # Pattern-tagged workout variants shared across zones
(race-pace tempo/reps today; fartlek/progression/decay/time-based land in Phase 2)
 %    %    % % %  workout-patterns.test.ts
 %    %    % % %  mixed-zone-workouts.ts   # Two-zone-blend workouts (E+M/M+T/T+I),
buildMixedZoneWorkouts
 %    %    % % %  mixed-zone-workouts.test.ts
 %    %    % % %  race-prep.ts             # 4-week race-prep plan — curates/relabels
buildZoneWorkouts output rather than a separate workout library, buildRacePrepResult,
isRacePrepEligible
 %    %    % % %  race-prep.test.ts
 %    %    % % %  segment-targets.ts       # Per-segment pace/power target-range math, shared by
the /workouts UI and fit-export.ts
 %    %    % % %  segment-targets.test.ts
 %    %    % % %  fit-export.ts            # Encodes a generated workout as a downloadable FIT
file (via @garmin/fitsdk), for watch upload
 %    %    % % %  fit-export.test.ts
 %    % % %  vite-plugins/
 %        % % %  git-dates.ts        # Vite plugin exposing virtual:git-dates — a per-route
lastmod date map derived from git history
 %        % % %  git-dates.test.ts
 % % %  routes/
     % % %  +layout.svelte   # Root layout — CookieBanner + header + main + SiteFooter
     % % %  +page.svelte     # Home page — HeroSection + ToolCard grid
     % % %  +error.svelte    # Error page
     % % %  pace/
     % % %  race-predictor/
     % % %  training-paces/
     % % %  hr-zones/
     % % %  vo2max/
     % % %  parkrun/
     % % %  power-zones/     # Power Zones Calculator (Stryd/Garmin/COROS*/Polar) — *COROS
currently hidden pending further research
     % % %  workouts/        # Workout Suggestions (Pace/Power/HR/Race-Prep modes, plus
Mixed-Zone Sessions) — includes the workout detail modal and "Download as .FIT" export
     % % %  privacy/         # Privacy Policy page
```

Design System

Tokens (`src/app.css`)

Dark mode is applied automatically via prefers-color-scheme, with an explicit html.light/html.dark class override (set pre-paint by app.html) taking priority once JS has run. Always use design tokens (text-ink, text-muted, bg-bg, border-ink/10) rather than hardcoded Tailwind colours where tokens exist.

Namespace note: color tokens must live under the --color-* prefix, not --text-* — Tailwind v4 reserves --text-* for font-size scale tokens (--text-sm, --text-lg, etc.), and a --text-muted-style color token silently compiles to an invalid font-size declaration instead of color, with no build error. (This was a real, previously-shipped bug — see PR #71.)

Text Contrast (WCAG AA)

Secondary text (help hints, labels, table headers, footer) uses .text-muted (!' --color-muted), which achieves ~4.63:1 contrast in light mode and ~6.92:1 in dark mode against the page background — both pass WCAG 2.1 AA (4.5:1 minimum). The dark-mode value is set once, at the token level, in src/app.css (html.dark and the no-JS prefers-color-scheme fallback) — never override it per-component.

An ESLint rule (runwise/no-low-contrast-text, in eslint-plugin-runwise/rules/no-low-contrast-text.js) errors on text-gray-400/text-gray-500 in Svelte files, including dark:/hover:-prefixed variants — there is no exemption for dark:text-gray-400 (it predates --color-muted's dark-mode fix and is no longer needed anywhere in the codebase).

Focus Rings (WCAG AA — SC 2.4.7)

All interactive elements (<button> and <a>) must have visible focus indicators for keyboard and assistive-technology users (WCAG 2.1 SC 2.4.7 Focus Visible).

Standard pattern:

```
  <!-- Buttons -->
  <button class="... focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-
accent focus-visible:ring-offset-2">

  <!-- Inline links (inside text) — add rounded-sm for clean ring shape -->
  <a href="..." class="rounded-sm ... focus-visible:outline-none focus-visible:ring-2
focus-visible:ring-accent focus-visible:ring-offset-2">
```

Ring offset note: Most elements use ring-offset-2. Tab buttons inside tight tablist containers use ring-offset-1 — choose whichever fits the layout.

Enforcement: The custom ESLint plugin eslint-plugin-runwise (in eslint-plugin-runwise/) provides two rules for Svelte files, both at 'error' severity in eslint.config.js:

```
  eslint-plugin-runwise/
  % % %  index.js                      # Plugin entry point
  % % %  rules/
      % % %  require-focus-visible.js   # Checks <button>/<a> for the full focus-visible
pattern
      % % %  no-low-contrast-text.js    # Bans text-gray-400/text-gray-500 (any variant) in
class attributes
```

Both rules operate on the Svelte template AST directly (SvelteElement, SvelteAttribute, SvelteLiteral) rather than plain ESTree nodes — Svelte class attributes are not standard Literal nodes, and the element name lives on SvelteElement.name.name with its attributes on SvelteElement.startTag.attributes, not on SvelteStartTag directly. Getting this AST path wrong makes a rule fail completely silently (it simply never matches, with zero lint errors either way) rather than throwing — if you write or modify a rule targeting Svelte templates, verify it actually fires against a deliberately-broken scratch .svelte file before trusting a clean npm run lint run.

Collapsible Content

Any field or block that toggles visibility based on user input (e.g. a "Custom" option revealing an extra input) should use the shared CollapsibleField component rather than a bespoke inline pattern:

```
  <CollapsibleField expanded={isCustom}>
  ”Ä–ç WDf–VÆB âââ óà
  </CollapsibleField>
```

Why not {#if} or the native hidden attribute:

- `{#if}` unmounts/remounts the content, so there's nothing to animate — the field would snap in/out instantly.
- The native `hidden` attribute also snaps instantly and sets no `aria-hidden`, so assistive technology gets no signal that the field is (or isn't) currently relevant.

What CollapsibleField does instead: the content stays mounted at all times (so its state — e.g. a partially-typed value — survives being hidden and re-shown) and visibility is purely a CSS transition (max-h-0 opacity-0 !” max-h-24 opacity-100 mb-4, overflow-hidden transition-all duration-200), paired with two accessibility attributes toggled together:

aria-hidden alone is not enough: it only affects the accessibility tree — it does not remove a nested focusable element from the tab order. Without inert, a sighted keyboard user can Tab into an invisible field (confirmed as a real, fixed bug — see PR #72 review, finding "aria-hidden properly set"). Always pair aria-hidden with inert (or tabindex="-1" on every focusable descendant, which doesn't scale) whenever hiding content that contains interactive elements.

inert's focus-blocking enforcement is real-browser behaviour that jsdom does not implement (it only reflects the IDL property, not the enforcement) — CollapsibleField.test.ts asserts the property, but the actual behavioural guarantee is covered by e2e/collapsible-field-focus.test.ts (Playwright/Chromium).

Hover Feedback (touch and mouse)

hover: classes must use the --color-hover token (.hover\:text-hover) rather than reusing .text-ink directly — this keeps the hover-state color relationship an explicit, named design decision instead of an incidental side effect of --color-ink flipping per theme.

Tailwind v4 wraps hover: utilities in @media (hover: hover) by default, to prevent "sticky hover" on touch devices (where a tap has no true hover-then-release gesture). This project overrides that default via a custom variant in src/app.css:

```
  @custom-variant hover (&:hover);
```

This makes every hover: utility sitewide apply on tap as well as mouse hover — chosen deliberately so that touch users get the same hover feedback as mouse users. The accepted trade-off: tapping an element leaves it visually "stuck" in its hover-colored state until another hoverable element is tapped. If you ever need the touch-safe default back for a specific element, use an explicit @media (hover: hover) wrapper on that element's own styles rather than reverting the global variant.

Components

Adding a New Tool

Create `src/routes/<tool-name>/+page.svelte`
Import `ToolLayout` and pass `title`, `description`, and `route` props
Add the tool to the `tools` array in `src/routes/+page.svelte`
Add the tool link to `SiteNav.svelte`
Register the route in `src/lib/seo.ts` (`PAGES` map) with a title, description, OG image path, and sitemap priority — this drives the page's meta tags, `sitemap.xml` entry, `robots.txt` allow rules, and JSON-LD structured data
Add tests alongside the page

Testing

Unit and component tests (Vitest)

```
npm run test          # Run all tests once
npm run test -- --watch  # Watch mode
```

Tests live alongside their source files (*.test.ts). Follow the existing pattern:

```
import { describe, it, expect, afterEach } from 'vitest';
import { render, cleanup, screen } from '@testing-library/svelte';
import MyComponent from './MyComponent.svelte';

afterEach(() => { cleanup(); });

describe('MyComponent', () => {
  it('renders correctly', () => {
    render(MyComponent, { props: { ... } });
    expect(screen.getByRole('...')).toBeInTheDocument();
  });
});
```

E2E tests (Playwright)

```
npx playwright install chromium  # First time only
npm run test:e2e                 # Run E2E tests against the production build
```

E2E tests live in the e2e/ directory. They run against the production preview server (npm run build && npm run preview on port 4173). Configuration is in playwright.config.ts.

Code Quality

```
npm run check        # TypeScript / svelte-check
npm run lint         # ESLint
npm run format       # Prettier (auto-fix)
npm run test:e2e     # Playwright E2E tests (requires built app)
```

Workflow

This project follows an issue-driven workflow:

```
/analyse <issue>   !’ requirements + acceptance criteria
```

```
/design <issue>    !’ architecture + work breakdown
/develop <issue>   !’ TDD implementation
/verify <PR>       !’ runtime verification
/pr-reviewer <PR>  !’ acceptance criteria audit
/merge <PR>        !’ merge + close issues
```